

CASE STUDY

Lumo

Make hard product decisions clearer, and keep a record of how they turned out.

ROLE

Solo build: product, design, engineering

BUILT WITH

Next.js 16, React 19, TypeScript, Claude

STATUS

Live prototype at www.trylumo.co

THE SHORT VERSION

Lumo is for product managers who have to make a hard call and then explain it to everyone it touches. You write down your first instinct in plain language. Lumo reads the situation back, surfaces options (including ones you may not have considered), and helps you commit with a written trade-off and a confidence rating. It then drafts a message for each audience that needs to hear the decision. The whole record is saved so the tool can later show you where your instincts have held up and where they have not.

I built it end to end as a work sample. It is a prototype, but a real one: live model calls, production deployment, English and Spanish.

AT A GLANCE

6

steps in one continuous path

2

of them stay human by design

3

AI routes, all schema-bound

0

sign-ups needed to start

01 The problem

*AI gave you more options.
It did not give you more judgment.*

Product managers still make hard calls the same way they always have: quickly, under pressure, and mostly alone. Then they spend days translating that call for leadership, engineering, sales, and customers. Generative AI made the drafting faster. It did not make the decisions better, and it never remembered how last quarter’s calls actually turned out.

Most tools start from zero every time. They help with the decision in front of you and then forget it. They rarely capture your first instinct before the analysis begins, and they almost never close the loop between what you believed and what happened.

WHAT PMS USE TODAY, AND WHERE IT BREAKS

TOOL	GOOD AT	FALLS SHORT
Chat assistant	Fast thinking on a single decision	Forgets it the moment the tab closes. No record of your first instinct.
Decision journal	Keeps an honest history	Entirely manual, so it is the first habit dropped in a busy week.
Docs and Slack	Easy to share with the team	Reasoning gets buried in threads. Outcomes never connect back to the call.
Frameworks (RICE, etc.)	Consistent scoring	Scores the options, not the judgment of the person choosing.

That gap is the problem Lumo is built around.

Not faster drafting. Not another answer engine. A tool that captures your instinct, structures the reasoning, and remembers how it turned out, so your judgment compounds instead of resetting with every decision.

02 Who it is for

A working product manager who regularly makes calls with real trade-offs: what to sequence, which date to promise, what to cut. Short on time, answerable to leadership, and stuck rewriting the same decision for different audiences.

A DECISION LUMO IS BUILT FOR

Your largest customer wants SSO by the end of Q3. Engineering says Q4 at the earliest. Sales has already hinted yes. Your VP wants an answer by Friday, and whatever you decide, four groups need to hear it in four different ways.

This example runs through the rest of this document.

PRINCIPLES I SET BEFORE WRITING ANY CODE

Capture the instinct first.

Ask for the raw take before any analysis, so there is something real to compare against later.

You commit, not the model.

The tool can structure, pressure-test, and draft. It does not choose.

Never fake it.

Real model output only. Sample content is clearly labeled as a sample.

Zero friction to start.

No account, no install, no approval from IT. Open the page and begin.

Interrupt the next mistake.

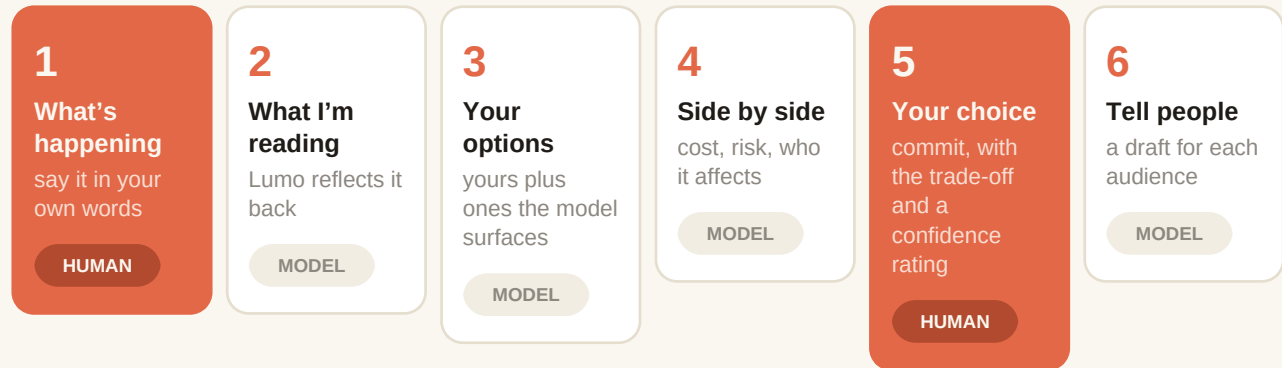
Surface a pattern at the moment someone is about to repeat it, not in a monthly report.

Fail honestly.

When the model or a service fails, say so plainly instead of showing a made-up result.

03 The six-step flow

The product is one continuous path. The model helps in the middle. The first and fifth steps stay human.



THE SSO DECISION, STEP BY STEP

STEP	WHAT THE PM DOES	WHAT LUMO RETURNS
1 What's happening	Types the situation and a first instinct: "Promise Q3, we can make it work."	Nothing yet. The instinct is saved before any analysis.
2 What I'm reading	Reads and corrects the framing.	The real question: protect the account or protect the roadmap. Who is affected. How pressing it is.
3 Your options	Adds an option of their own.	Options such as a phased SSO beta in Q3 with full rollout in Q4.
4 Side by side	Compares cost, risk and reversibility.	A comparison of each option, including who absorbs the cost.
5 Your choice	Picks phased beta, writes the trade-off, rates confidence 3 of 5.	Nothing. The choice is the PM's alone.
6 Tell people	Edits and sends.	Drafts for the VP, engineering, sales and the customer, each with likely pushback and a reply.

Over time the system can say something like: you have been this sure about a date ten times. Seven of them slipped.

That is the point. Not a better answer in the moment, but a clearer picture of your own judgment, built up over real decisions.

04 The decision that mattered most

When I first built Lumo, it recommended a decision. That seemed like the obvious useful feature. Within a couple of weeks I realized I hated it. The moment the model told me what it would do, I stopped thinking and started reacting. The tool had quietly shifted from helping me reason to giving me an answer I could accept or reject. That was the opposite of the problem I was trying to solve.

So I removed the recommendation completely.

Claude can structure the situation, surface options I might have missed, and draft the follow-ups. It cannot make the call. Having to choose is the point of the product.

THE TRADE-OFF I ACCEPTED

Removing the recommendation made the product feel less impressive in a quick demo. A tool that hands you an answer looks smarter in the first thirty seconds. I chose the version that is more useful on the fifth decision over the one that is more impressive on the first.

05 Other choices I made on purpose

Your instinct goes in first.

Step 1 captures it before Lumo says anything. Without that record there is nothing to learn from later, and the model's framing would quietly overwrite it.

Committing is deliberate.

Choosing takes a press-and-hold, so it never happens by accident. There is an instant alternative for keyboard and assistive-technology users.

One decision, many voices.

The VP, the team, other teams and customers each get a draft in the right channel (email, Slack or DM), plus the pushback they are likely to raise and a reply.

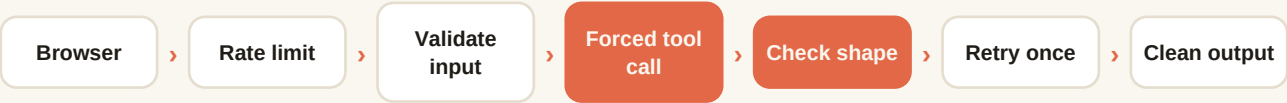
Restraint over polish.

Calm tone, little motion, one serif signature line used sparingly. No exclamation points. A tool for stressful moments should not add noise.

06 How it works

Nothing exotic. The effort went into making the AI predictable: the model answers through a schema, and the server checks every response before it reaches the screen.

EVERY AI REQUEST TAKES THE SAME PATH



Highlighted stages are where most of the reliability comes from. If both attempts fail, the user sees a plain-language error and the option to view a labeled example instead.

STACK

LAYER	CHOICE	WHY
App	Next.js 16, React 19, TypeScript	Fast, indexable pages and strict types for every AI payload.
AI	Claude with forced tool calls	Structured output only, never free text the UI has to guess at.
Guardrails	Server-side validation and one retry	Bad output is rejected, not shown.
Storage	Browser only, no sign-up	Zero friction, and decisions stay on the user’s device.
Safety	Upstash Redis rate limiting	20 requests per hour per route keeps a public tool affordable.
Reach	English and Spanish on Vercel	The model replies in the visitor’s language with its own writing rules.

THREE API ROUTES

/api/analyze

Reads the situation back and generates options with cost, risk and reversibility.

/api/decide

Turns a committed decision into a summary and a draft for every audience.

/api/rewrite

Rewrites a single draft on request without redoing the whole decision.

07 A closer look at the code

Two excerpts from the route that drafts audience messages. They are trimmed for length but otherwise as written.

CODE app/api/decide/route.ts (schema, abridged)

```
const decideTool: Anthropic.Tool = {
  name: "provide_audience_drafts",
  input_schema: { type: "object", properties: {
    claritySummary: { type: "string" },
    drafts: { type: "array", minItems: 2, maxItems: 8, items: {
      properties: {
        // "vp" is required so leadership is never left out
        audienceId: { type: "string" },
        channel: { type: "string", enum: ["email", "slack", "dm"] },
        subject: { type: "string" }, body: { type: "string" },
        pushback: { type: "array", maxItems: 2 /* objection + reply */ },
      },
      additionalProperties: false,
    } },
  } },
}
```

WHY IT MATTERS The model cannot reply in free text. Every field the UI renders is declared up front, so an empty or malformed draft is a detectable error rather than a confusing screen.

CODE app/api/decide/route.ts (retry loop, abridged)

```
for (let attempt = 1; attempt <= 2 && !result; attempt++) {
  const response = await anthropic.messages.create({
    model: ANTHROPIC_MODEL,
    tools: [decideTool],
    // Force the structured tool; plain text is not an option
    tool_choice: { type: "tool", name: decideTool.name },
    // ...system prompt, user prompt, max_tokens
  })
  const toolResult = response.content.find(isDecideToolUse)
  if (!toolResult) { record("no tool output"); continue }
  // Schema is a request; this check is the guarantee
  if (!isDecideResult(toolResult.input, "vp")) { record("bad shape"); continue }
  result = toolResult.input
}
if (!result) return getDiagnosticsResponse(attempts, lastFailure)
```

WHY IT MATTERS Schemas reduce bad output but do not eliminate it. A second, independent check on the server, plus one retry, turned occasional broken screens into rare, clearly labeled errors.

RATE LIMITING THAT FAILS OPEN

CODE lib/rate-limit.ts (abridged)

```
decide: new Ratelimit({
  redis,
  limiter: Ratelimit.slidingWindow(20, "1 h"),
  prefix: "lumo:ratelimit:decide",
}),

try {
  const { success, reset } = await limiters[route].limit(ip)
  if (success) return { allowed: true }
  return { allowed: false, retryAfter: secondsUntil(reset) }
} catch (error) {
  // A limiter outage must never take the product down with it
  console.error(`[rate-limit] Redis check failed for "${route}"`, error)
  return { allowed: true }
}
```

WHY IT MATTERS A public AI tool needs a cost ceiling, but the limiter is a safeguard, not the product. I chose to fail open: if Redis is unreachable, the request goes through and the failure is logged.

08 How I tested whether it actually helped

I tested on real decisions rather than invented ones, and watched for three things:

- Did people change their initial instinct after seeing the structured options, or just confirm what they already believed?
- Did the confidence rating create useful friction, or did users treat it as a formality?
- Where did the model produce options that sounded reasonable but were shallow or misframed?

The pattern that showed up most often: the model is good at expanding the set of options and drafting clear messages. It is less reliable at surfacing the non-obvious downside or the political constraint a human already feels but has not named. When the tool was most useful, it was usually because it forced the person to write the trade-off down, not because it had a better idea.

09 What broke in production

Three bugs that only appeared on the live site, and what each one taught me.

The flow reset itself mid-step.

An unstable React key remounted the current step on every update, wiping the user's input. I fixed the key, then re-ran all six steps in a real browser.

AI worked in preview, not live.

The production runtime needed explicit model configuration. I centralized it in one place and added visible AI status and logging.

The sitemap returned 404.

Vercel redirected the bare domain to www and my middleware redirected it back. I made www the single canonical host.

The lesson

Test the deployed product, not the preview. Make failures visible. Treat domains and redirects as product work, because a search engine that cannot read the sitemap is a user who cannot find you.

10 What still does not work well

The model sometimes offers options that are plausible on the surface and weak on inspection. It can miss organizational reality: the stakeholder who will block the decision, the commitment made quietly, the constraint that only exists in people's heads. Confidence ratings are easy to rush.

And because everything is stored in the browser, the longer-term pattern detection is still mostly illustrative rather than proven on real longitudinal data. These are the next problems the product has to confront if it is going to be trusted with real decisions.

11 What I would build next

#	NEXT STEP	WHY
1	Outcome check-ins	Ask at a set date how the decision actually landed. That is the data the whole product depends on.
2	Pattern detection on real history	Replace the sample record with the user's own decisions.
3	Better handling of weak options	Make it fast to flag and dismiss a shallow suggestion, and learn from it.
4	Light team context	Account for known constraints without building a full knowledge base.
5	Accounts, then Slack and email	Keep the record across devices and send drafts where people already work.

12 What this project shows about how I work

I start from the problem, not the model.

The core design choice was taking a capability away. The product is defined by what the AI is not allowed to do.

I ship and own the whole thing.

Product thinking, interface, prompts, API design, deployment, SEO and localization, carried through to a live domain.

I treat AI output as untrusted input.

Schemas, validation, retries and honest error states are product features, not afterthoughts.

I am honest about limits.

I can say clearly where the product works, where it does not yet, and what would prove it.

*Lumo is my attempt at the part most AI tools still skip:
better judgment, and a memory of how it turned out.*

Terrance Range | www.trylumo.co